

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Сибирский государственный университет телекоммуникаций и
информатики»

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

Отчет по производственной практике
Занятие №2

по теме:

ВВЕДЕНИЕ В АРХИТЕКТУРУ SDR-УСТРОЙСТВ. ЗНАКОМСТВО С
БИБЛИОТЕКАМИ SOAPY SDR, LIBIO ДЛЯ РАБОТЫ С ADALM PLUTO
SDR. ИНИЦИАЛИЗАЦИЯ SDR-УСТРОЙСТВА. РАБОТА С БУФЕРОМ:
ПОЛУЧЕНИЕ ЦИФРОВЫХ IQ-ОТСЧЕТОВ.

Студент:
Группа ИА-331

Я.А Гмыря

Предподаватели:
Лектор
Семинарист
Семинарист

Калачиков А.А
Ахнашев А.В
Попович И.А

Новосибирск 2025 г.

СОДЕРЖАНИЕ

1	ВВЕДЕНИЕ	3
1.1	Введение	3
1.2	Цель	3
2	ЛЕКЦИЯ	4
2.1	Архитектура простого приемника	4
2.2	Принцип работы демодулятора	4
3	ПРАКТИЧЕСКАЯ ЧАСТЬ	7
3.1	Передача данных между Adalm Pluto и хостом	7
3.2	Timestamping	7
3.3	Установка необходимых библиотек и зависимостей	7
3.3.1	SoapySDR	7
3.3.2	Libiio	8
3.3.3	LibAD9361	9
3.3.4	SoapyPlutoSDR	9
3.4	Основные моменты работы с Adalm Pluto напрямую из C++	10
3.4.1	Подключение библиотек	10
3.4.2	Инициализация устройства	10
3.4.3	Формирование потоков и буферов	10
3.4.4	Получение I/Q семплов	11
3.4.5	Освобождение памяти	11
3.5	Результат работы	12
4	ВЫВОД	13

ВВЕДЕНИЕ

1.1 Введение

На прошлом занятии мы работали с SDR с помощью программы GNU Radio, но для более гибкой работы она не очень подходит, поэтому с этого занятия мы начнем работать с SDR с помощью C/C++ и библиотеки SoapySDR. Также на прошлом занятии мы рассмотрели архитектуру передатчика, поэтому на этом занятии познакомимся с архитектурой приемника.

1.2 Цель

Познакомиться с упрощенной архитектурой приемника. Научиться работать с SDR напрямую из C/C++ с помощью библиотеки SoapySDR. Отправить в эфир семплы, а потом принять их и проанализировать.

ЛЕКЦИЯ

2.1 Архитектура простого приемника

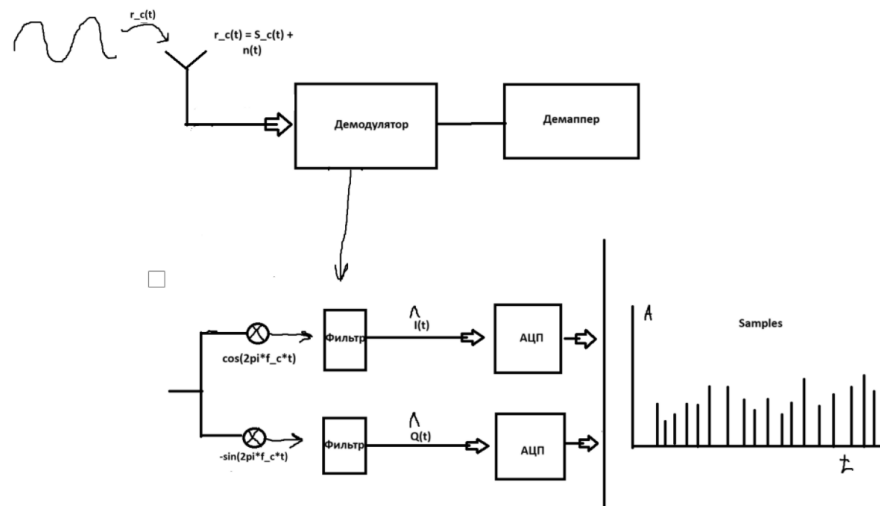


Рисунок 1 — Архитектура простого приемника

Сигнал поступает на антенну приемника и преобразуется в колебание электрического тока. Первым делом нам нужно выделить из этого высокочастотного сигнала сигнал низкой частоты, т.к с низкочастотным сигналом работать проще и вся информации содержится именно в нем. Для этого подаем сигнал на демодулятор

Демодуляция - процесс, обратный модуляции колебаний, выделение информационного (модулирующего) сигнала из модулированного колебания высокой (несущей) частоты. При передаче цифровых сигналов в результате демодуляции получается последовательность символов, передающих исходную информацию ($I(t)$ и $Q(t)$).

2.2 Принцип работы демодулятора

На схеме видим, что поступивший в демодулятор сигнал проходит через цепь, в которой он перемножается на несущие $\cos(\omega_c t)$ и $-\sin(\omega_c t)$. Но как это помогает нам узнать низкочастотный сигнал?

Вспомним из прошлого занятия, какой сигнал по итогу мы отправляли в радиоканал:

$$s(t) = I(t) \cos(2\pi f_c t) - Q(t) \sin(2\pi f_c t)$$

Рисунок 2 — Отправляемый сигнал

Вспомним тригонометрические формулы

Тема: Произведение синусов и косинусов.

$$\sin \alpha \cdot \sin \beta = \frac{1}{2} (\cos(\alpha - \beta) - \cos(\alpha + \beta))$$

$$\cos \alpha \cdot \cos \beta = \frac{1}{2} (\cos(\alpha - \beta) + \cos(\alpha + \beta))$$

$$\sin \alpha \cdot \cos \beta = \frac{1}{2} (\sin(\alpha - \beta) + \sin(\alpha + \beta))$$

Рисунок 3 — Тригонометрические формулы

Теперь произведем перемножения входного сигнала на несущие:

$$\begin{aligned} & (I(t)\cos(\omega_c t) - Q(t)\sin(\omega_c t)) * \cos(\omega_c t) \\ &= I(t)\cos(\omega_c t)\cos(\omega_c t) - Q(t)\sin(\omega_c t)\cos(\omega_c t) \\ &= \frac{I(t)}{2}(\cos(2\omega_c t) + \cos(0)) - \frac{Q(t)}{2}(\sin(2\omega_c t) + \sin(0)) \\ &= \frac{I(t)}{2}\cos(2\omega_c t) - \frac{Q(t)}{2}\sin(2\omega_c t) + \boxed{\frac{I(t)}{2}} \end{aligned}$$

У нас получилось выделить синфазную компоненту $I(t)$.

Прделаем те же действия с умножением на $\sin$

$$\begin{aligned} & (I(t) \cos(\omega_c t) - Q(t) \sin(\omega_c t)) \cdot \sin(\omega_c t) \\ &= I(t) \cos(\omega_c t) \sin(\omega_c t) - Q(t) \sin^2(\omega_c t) \\ &= \frac{I(t)}{2} \sin(2\omega_c t) - \frac{Q(t)}{2} (1 - \cos(2\omega_c t)) \\ &= \frac{I(t)}{2} \sin(2\omega_c t) + \frac{Q(t)}{2} \cos(2\omega_c t) - \boxed{\frac{Q(t)}{2}} \end{aligned}$$

Получили квадратурную компоненту $Q(t)$.

Помимо самих компонент остались и другие сигналы, которые нам не нужны, поэтому с помощью фильтра уберем их. На выходе получим чистые $\frac{I}{2}$ и $-\frac{Q}{2}$. Можно заметить, что после извлечения символов их амплитуда упала вдвое. Эта проблема решается путем усиления (на схеме не отображено)

Далее компоненты поступают на АЦП, где будут ”нарезаны” на семплы. В этих семплах нужно произвести символьную синхронизацию, чтобы правильно выделить переданную информацию, но это тема следующих занятий.

ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1 Передача данных между Adalm Pluto и хостом

Передача данных (IQ-сэмплов) между Adalm Pluto и хост-компьютером осуществляется посредством USB 2.0. Важно отметить, что в случае с SDR, данные необходимо передавать непрерывно в обе стороны (с хост-компьютера на SDR и обратно) одновременно. Хотя и теоретическая пропускная способность USB 2.0 равна 480 Mb/s, работа в полудуплексном режиме с передачей данных в обе стороны одновременно (с точки зрения пользователя) разительно снижается. Целевое значение частоты дискретизации желательно задавать в пределах 6 Msps.

3.2 Timestamping

В библиотеке SoapySDR реализованы функции получения временных меток (timestamp) с FPGA (Xilinx Zynq). Временные метки (timestamp) привязаны к каждому запросу данных с буфера ПЛИС, что, в свою очередь, позволяет синхронно получать/передавать данные в потоках RX/TX. Более того, из-за проблем с пропускной способностью USB 2.0 возникает проблема увеличения частоты дескритизации, при больших значениях которой, USB 2.0 не может обеспечить полноценную передачу и прием (одновременных) сэмплов из Adalm Pluto на хост-компьютер. Выявить данную проблему можно благодаря реализации функции временных меток с Xilinx Zynq.

3.3 Установка необходимых библиотек и зависимостей

3.3.1 SoapySDR

SoapySDR — открытая обобщённая API и библиотека времени выполнения для взаимодействия с SDR-устройствами. С помощью SoapySDR можно создавать экземпляры, настраивать и вести потоковую передачу данных с SDR-устройством в различных средах. Большинство готовых SDR-платформ поддерживаются SoapySDR, и многие открытые приложения используют

SoapySDR для интеграции с оборудованием. Кроме того, SoapySDR имеет привязки к средам разработки, таким как GNU Radio и Pothos.

```
sudo apt-get install python3-pip python3-setuptools
sudo apt-get install cmake g++ libpython3-dev python3-numpy swig
python3-matplotlib

git clone --branch soapy-sdr-0.8.1
    https://github.com/TelecomDep/SoapySDR.git

cd SoapySDR
mkdir build && cd build

cmake ../

make -j 16
sudo make install
sudo ldconfig
```

3.3.2 Libiio

libiio — библиотека, разработанная компанией Analog Devices, которая предназначена для упрощения работы с устройствами ввода-вывода данных (I/O), особенно с программируемыми аналогово-цифровыми и цифро-аналоговыми преобразователями (ADC/DAC), а также с радиооборудованием на базе платформы ADI (например, ADALM-PLUTO). Позволяет читать и записывать данные в реальном времени.

```
sudo apt-get install libxml2 libxml2-dev bison flex libcdk5-dev
cmake
sudo apt-get install libusb-1.0-0-dev libaio-dev pkg-config
sudo apt install libavahi-common-dev libavahi-client-dev

git clone --branch v0.24 https://github.com/TelecomDep/libiio.git

cd libiio
mkdir build && cd build
cmake ../
make -j 16
sudo make install
```

3.3.3 LibAD9361

LibAD9361 - библиотека для работы с радиочипами семейства AD9361 от Analog Devices. В сочетании с libiio позволяет организовать потоковое чтение/запись данных в реальном времени.

```
git clone --branch v0.3
    https://github.com/TelecomDep/libad9361-iio.git
cd libad9361-iio

mkdir build && cd build

cmake ../

make -j 16
sudo make install
sudo ldconfig
```

3.3.4 SoapyPlutoSDR

SoapyPlutoSDR - библиотека, которая является расширением библиотеки SoapySDR, предназначенная для работы конкретно с Adalm Pluto.

```
git clone --branch sdr_gadget_timestamping
    https://github.com/TelecomDep/SoapyPlutoSDR.git
cd SoapyPlutoSDR

mkdir build && cd build

cmake ../

make -j 16
sudo make install
sudo ldconfig
```

3.4 Основные моменты работы с Adalm Pluto напрямую из C++

3.4.1 Подключение библиотек

```
// Init device
#include <SoapySDR/Device.h>
// Data types for writing samples
#include <SoapySDR/Formats.h>
```

3.4.2 Инициализация устройства

```
//create struct for init
SoapySDRKwargs args = {};

//Select device type
SoapySDRKwargs_set(&args, "driver", "plutosdr");
if (1) {
    // Sample transmission method (usb)
    SoapySDRKwargs_set(&args, "uri", "usb:");
} else {
    // Or IP
    SoapySDRKwargs_set(&args, "uri", "ip:192.168.2.1");
}
SoapySDRKwargs_set(&args, "direct", "1");
// Buffer size and timestamps
SoapySDRKwargs_set(&args, "timestamp_every", "1920");
SoapySDRKwargs_set(&args, "loopback", "0");
// Init
SoapySDRDevice *sdr = SoapySDRDevice_make(&args);
// Free memory
SoapySDRKwargs_clear(&args);
```

3.4.3 Формирование потоков и буферов

```
// create streams
SoapySDRStream *rxStream = SoapySDRDevice_setupStream(sdr,
    SOAPY_SDR_RX, SOAPY_SDR_CS16, channels, channel_count, NULL);
SoapySDRStream *txStream = SoapySDRDevice_setupStream(sdr,
    SOAPY_SDR_TX, SOAPY_SDR_CS16, channels, channel_count, NULL);
```

```

//start streaming
SoapySDRDevice_activateStream(sdr, rxStream, 0, 0, 0);
SoapySDRDevice_activateStream(sdr, txStream, 0, 0, 0);

// Get RX/TX MTU sizes

size_t rx_mtu = SoapySDRDevice_getStreamMTU(sdr, rxStream);
size_t tx_mtu = SoapySDRDevice_getStreamMTU(sdr, txStream);

// allocate memory for buffers (for RX/TX samples)
int16_t tx_buff[2 *tx_mtu];
int16_t rx_buffer[2 *rx_mtu];

```

3.4.4 Получение I/Q семплов

```

// start receive samples
for (size_t buffers_read = 0; buffers_read < iteration_count;
    buffers_read++)
{
    void *rx_buffs[] = {rx_buffer};
    // flags set by receive operation
    int flags;
    //timestamp for receive buffer
    long long timeNs;

    // Read samples from stream and write I/Q samples in file
    int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs,
        rx_mtu, &flags, &timeNs, timeoutUs);
    // write in file
    for(int i = 0; i < rx_mtu * 2; i++){
        fprintf(file, "%d %d\n", rx_buffer[i], rx_buffer[i+1]);
    }
}

```

3.4.5 Освобождение памяти

```

//stop streaming

```

```
SoapySDRDevice_deactivateStream(sdr, rxStream, 0, 0);
SoapySDRDevice_deactivateStream(sdr, txStream, 0, 0);

//shutdown the stream
SoapySDRDevice_closeStream(sdr, rxStream);
SoapySDRDevice_closeStream(sdr, txStream);

//cleanup device handle
SoapySDRDevice_unmake(sdr);
```

3.5 Результат работы

После работы программы создается файл `samples.txt`, в котором будут храниться полученные семплы в формате: (I,Q). Для визуализации $I(t)$ и $Q(t)$ воспользуемся Python.

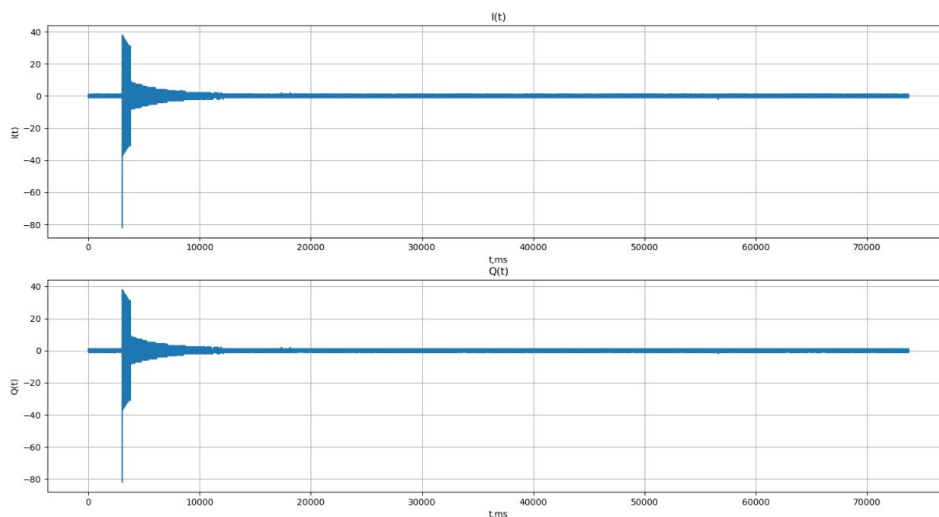


Рисунок 4 — Графики $I(t)$ и $Q(t)$

Можем заметить, что графики напонируют прямоугольный сигнал.

ВЫВОД

В ходе проделанной работы я познакомился с архитектурой простого приемника. Научился работать с Adalm Pluto SDR напрямую из C/C++ с помощью библиотеки SoapySDR. Путем визуализации семплов узнал, как выглядит сигнал в виде дискретных отсчетов.